

COMILLAS
UNIVERSIDAD PONTIFICIA
ICAI ICADE CIHS

Sprint 2

Grupo:

Jorge Carnicero Príncipe,

Andrés Gil Vicente,

Jorge González Pérez

Tecnologías de Procesamiento Big Data

3º Grado en Ingeniería Matemática e Inteligencia Artificial

Índice

Índice **2**
Introducción **3**
Objetivo del Escenario 2 3
Alcance del Sprint 2 3
Metodología **4**
Arquitectura del Escenario 4
Proceso de Despliegue 4
Implementación del Algoritmo de Montecarlo 6
Descripción del Código 6
Configuración de Pruebas 8
Herramientas de Monitorización 9
Resultados **10**
Datos Experimentales 10
Análisis de Rendimiento 11
Comportamiento del Sistema 17
Conclusión **19**
Comparativa Escenario 1 vs Escenario 2 19
- Diferencias del Framework 19
- Diferencias en la Orquestación 20
- Diferencias en el Enfoque de Despliegue 20
Limitaciones Observadas 20
Reflexión y conclusión final 21

Introducción

Este informe documenta el desarrollo y análisis del segundo escenario del proyecto BENCHDATA, correspondiente a la implementación de un clúster distribuido basado en Dask, un framework de computación paralela flexible diseñado específicamente para escalar las bibliotecas del ecosistema PyData (NumPy, Pandas, Scikit-learn) en entornos distribuidos.

Objetivo del Escenario 2

El Escenario 2 tiene como propósito desplegar y evaluar un clúster Dask utilizando contenedores Docker sobre instancias AWS EC2. Dask ha sido seleccionado por su integración nativa con el ecosistema científico de Python, su capacidad de escalado dinámico y su modelo de programación familiar para usuarios de NumPy y Pandas.

Para realizar una evaluación cuantitativa del rendimiento y permitir una comparación objetiva con el Escenario 1 (Ray), se ha implementado el mismo algoritmo de cálculo del número π mediante el método de Montecarlo, manteniendo idéntica la metodología.

Alcance del Sprint 2

Durante este segundo sprint, se han llevado a cabo las siguientes actividades principales:

- Construcción y despliegue de contenedores Docker para el clúster Dask
- Desarrollo e implementación del algoritmo de Montecarlo distribuido con Dask
- Ejecución de pruebas de rendimiento con diferentes configuraciones y datos
- Análisis y documentación de resultados
- Comparación preliminar con los resultados del Escenario 1

Este segundo escenario permite evaluar las capacidades de Dask como alternativa tecnológica para el entorno de desarrollo independiente propuesto, facilitando tanto la comparación homogénea con Ray, como el posterior análisis que se hará con Hadoop en el Sprint 3.

Contexto del Proyecto

Este razonamiento es el mismo que se explicó en el Sprint 1. En el contexto empresarial actual, las organizaciones se enfrentan al desafío de gestionar entornos de producción compartidos que deben soportar simultáneamente procesos productivos y tareas de desarrollo. Esta situación genera frecuentes saturaciones de recursos que comprometen tanto el rendimiento como la estabilidad del sistema. El proyecto BENCHDATA surge como respuesta a esta problemática, con el objetivo de evaluar diferentes arquitecturas distribuidas que permitan crear un entorno de desarrollo independiente, estable, escalable y accesible para los equipos de Data Science.

Metodología

Arquitectura del Escenario

El Escenario 2 se basa en una arquitectura distribuida de tipo Scheduler-Worker implementada mediante Dask, donde un nodo planificador (Dask Scheduler) coordina la ejecución de tareas distribuidas entre múltiples nodos trabajadores (Dask Workers). Esta arquitectura se ha desplegado utilizando Docker Compose, garantizando reproducibilidad, portabilidad y facilidad de gestión del entorno, además de mayor comodidad para poder modificar el `docker-compose.yaml` cuando se ha necesitado para aumentar o reducir el número de nodos workers.

La infraestructura levantada se compone de los siguientes elementos:

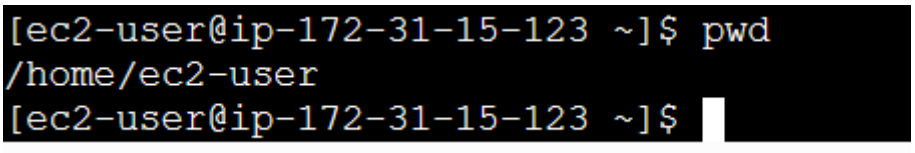
- Dask Scheduler: Nodo principal que gestiona la planificación de tareas entres los nodos workers, mantiene el estado del clúster, registra información de los workers disponibles y proporciona el dashboard de monitorización en tiempo real.
- Dask Workers: Nodos trabajadores (configurables de 1 a 3 mediante el archivo `docker-compose.yaml`) que ejecutan las tareas asignadas por el Scheduler. Cada worker gestiona su propia memoria y recursos computacionales.
- Instancia EC2: Máquina virtual en AWS (tipo r8g.large con 2 vCPUs) que aloja todos los contenedores Docker del clúster.

Proceso de Despliegue

El despliegue del Escenario 2 se ha realizado siguiendo un proceso estructurado en varias fases, que es prácticamente idéntico al que tuvimos que realizar en el Escenario 1:

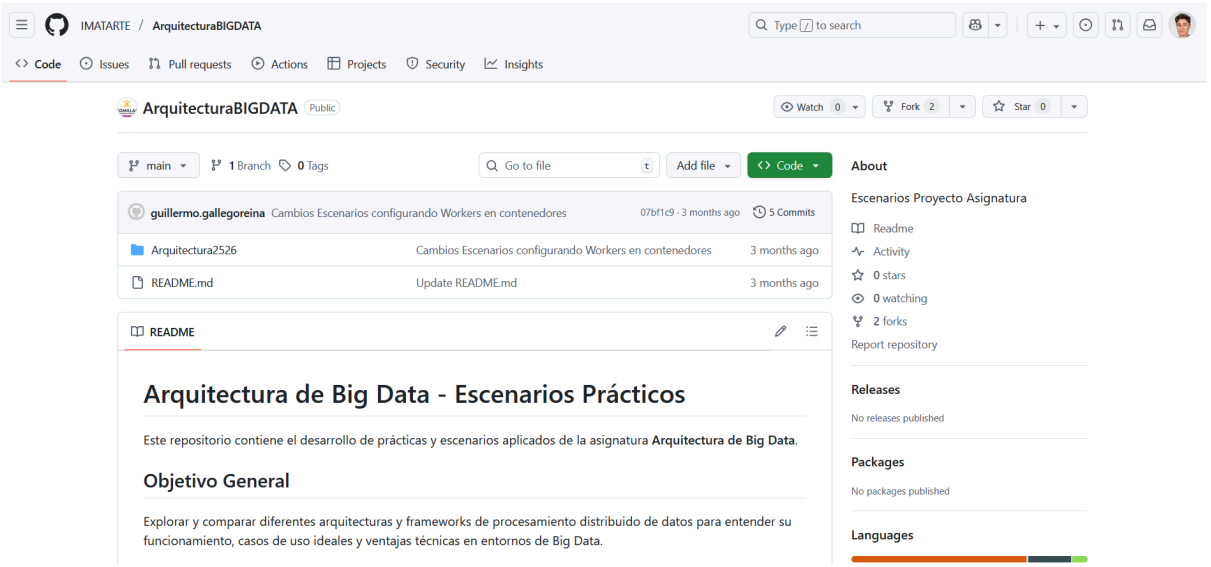
1. Preparación del Entorno

   Acceso a la instancia EC2: Al igual que en el escenario anterior, se ha establecido conexión SSH con la instancia EC2 previamente configurada. La conexión se realizó utilizando la clave SSH proporcionada y verificando el acceso al directorio raíz: `/home/ec2-user`.

   Clonación del repositorio: Una vez establecida la conexión, utilizamos el mismo repositorio que ya clonamos en el Escenario 1 con:

   ```
   git clone https://github.com/IMATARTE/ArquitecturaBIGDATA.git
   ```



Posteriormente se navegó hasta el directorio específico del Escenario 2:

```
cd ArquitecturaBIGDATA/Arquitectura2526/Practica/Escenario2
```

```
[ec2-user@ip-172-31-15-123 home]$ ls
ec2-user
[ec2-user@ip-172-31-15-123 home]$ cd ec2-user/ArquitecturaBIGDATA/Arquitectura2526/Practica/Escenario2
[ec2-user@ip-172-31-15-123 Escenario2]$ ls
docker-compose.yaml  sprint2.py  start.sh  stop.sh
```

2. Construcción e Inicialización de Contenedores

La configuración de permisos de Docker y la construcción de imágenes no se tuvo que repetir, ya que con haberlo hecho anteriormente en el Escenario 1 ya era suficiente. Recordemos que los comandos eran:

```
sudo gpasswd -a $USER docker
```

```
newgrp docker
```

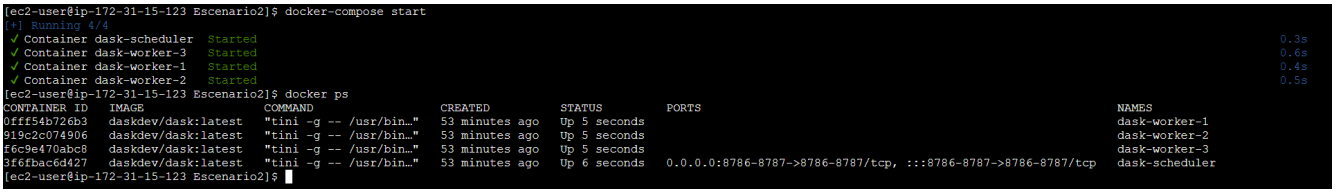
```
docker-compose build --no-cache
```

Inicialización del clúster: Lo que sí que tuvimos que hacer, de igual forma que se había hecho en el Escenario 2 y siempre que se tengan que levantar nuevos contenedores, es el hecho de iniciar los contenedores en modo detached (segundo plano), mediante el comando:

```
docker-compose up -d
```

Verificación del despliegue: Una vez ya hemos levantado los contenedores de Dask, siempre conviene verificar que el proceso se ha completado correctamente, para ello podemos ejecutar el comando:

```
docker ps
```



De esta forma vemos que tenemos levantados 4 contenedores, 1 para el Head Node o Work Scheduler, y otros 3 para cada uno los Worker Nodes, que son los que se necesitan para la primera de las pruebas en este escenario. Posteriormente se repetirá el proceso con 2 y 1 workers.

Implementación del Algoritmo de Montecarlo

Fundamento Matemático: el método de Montecarlo que hemos empleado para calcular π se basa en la relación geométrica entre un cuadrado y un círculo inscrito.

Al generar puntos aleatorios uniformemente distribuidos en un cuadrado de lado 1 y contar cuántos caen dentro del círculo de radio 1, podemos estimar π mediante la siguiente fórmula:

$$\pi \approx 4 \times (\text{puntos dentro del círculo} / \text{total de puntos})$$

Este método es ideal para evaluación de sistemas distribuidos porque:

- Es altamente paralelizable (cada iteración es independiente)
- La carga computacional es ajustable (variando el número de puntos)
- Proporciona una métrica clara de precisión (comparando con el valor real de π)

Descripción del Código

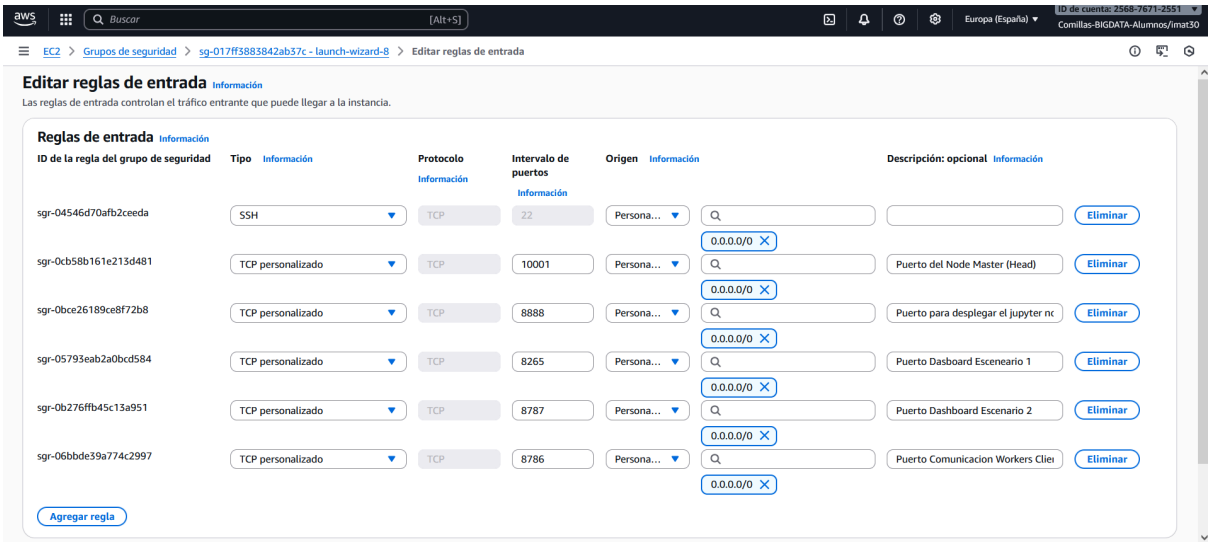
El script `escenario2.py` implementa una versión distribuida del algoritmo utilizando las capacidades de Dask. La lógica del código tanto a nivel de cálculo de π, como a nivel de computación distribuída son iguales que las que se emplearon en el Escenario 1 con Ray.

Lo único que cambia en este escenario es la sintaxis y funciones empleadas para distribuir las tareas, ya que ahora estamos utilizando Dask:

Configuración de la conexión:

```
client = Client("tcp://localhost:8786")
```

Nos conectamos mediante el puerto 8786, puerto el cual hemos tenido que configurar desde el firewall para poder comunicarnos correctamente con los workers y clientes de Dask:



Cabe destacar además, que para conectarnos al Client, tenemos que usar el protocolo TCP. La función principal de este tipo de protocolo es garantizar que los datos que se envían por una red lleguen de forma correcta, ordenada y completa al destino.

Función distribuida:

```
def montecarlo_pi_in_circle(puntos,seed_gen) -> int:
    np.random.seed(seed_gen)
    x,y = np.random.random(puntos),np.random.random(puntos)
    inside_circle = (x**2 + y**2) <= 1
    return int(inside_circle.sum())
```

En este caso es necesario el seed_gen, ya que el inconveniente que nos presentaba Dask en este caso era que las tareas distribuídas se nos ejecutan con la misma semilla de forma que si no la cambiamos de alguna manera, a la hora de distribuir las tareas obtenemos los mismos números aleatorios. Esto hace que no consigamos el propósito deseado, y dejaría de tener sentido distribuir las tareas ya que todas ellas tendrían el mismo resultado puesto que se ejecutan con el mismo número aleatorio y la misma cantidad de puntos.

No obstante, usando una semilla que se pasa externamente mediante un parámetro distinto para cada una de las tareas, solucionamos el problema mencionado.

Ejecución paralela:

```
NUM_TAREAS = 200

for puntos in [x * 1000000 for x in [5, 10, 50]]:

    puntos_por_tarea = puntos // NUM_TAREAS

    futuros = [client.submit(montecarlo_pi_in_circle,
    puntos_por_tarea,np.random.randint(puntos))for _ in
    range(NUM_TAREAS)]

    puntos_dentro = client.gather(futuros)

    pi = 4 * sum(puntos_dentro) / puntos
```

Se lanzan 200 tareas independientes para cada configuración.

Cada tarea ejecuta una simulación de Montecarlo con una parte de los puntos totales (`puntos // 200 (NUM_TAREAS)`), y calcula cuántos de esos puntos caen dentro del círculo unidad.

Finalmente, se combinan los resultados de todas las tareas (usando `client.gather`) para estimar el valor de π mediante la expresión:

$$\pi \approx 4 \times \frac{\text{puntos dentro del círculo}}{\text{puntos totales}}$$

Configuración de Pruebas

Siguiendo las especificaciones del enunciado del proyecto, se han evaluado las siguientes configuraciones:

Número de Workers: 1, 2, 3

- Nos permite analizar cómo escala el sistema al añadir más nodos de procesamiento, viendo si siempre hay mejoras o si hay algún cuello de botella.

Número de Puntos para aproximar el valor de π: 5.000.000, 10.000.000, 50.000.000

- Estas cargas representan diferentes intensidades computacionales, desde tareas ligeras hasta otras más pesadas.

Combinaciones totales: 9 configuraciones (3 workers × 3 cargas)

Para cada configuración se ha ejecutado el script `escenario2.py` completo, lanzando 200 tareas paralelas. Cada tarea calcula cuántos puntos, de un subconjunto del total, caen dentro del círculo unidad. Posteriormente, se suman los resultados de todas las tareas y, a partir del total de puntos generados, se obtiene una única estimación de π y registramos las siguientes métricas:

1. Tiempo de ejecución total (segundos): Desde el lanzamiento de las tareas hasta la recopilación de todos los resultados.

2. Velocidad de procesamiento (puntos/segundo): Calculada como el número total de puntos procesados dividido por el tiempo de ejecución.

3. Estimación de π: Valor obtenido a partir de la suma de los resultados de las 200 tareas, cada una aportando su parte.

Herramientas de Monitorización

Durante las ejecuciones, se utilizó el dashboard de Dask para monitorizar en tiempo real:

- Utilización de CPUs y memoria en cada worker
- Distribución de carga entre nodos workers
- Posibles cuellos de botella o errores
- Información detallada sobre transferencia de datos entre scheduler y workers

El dashboard es accesible a través del puerto 8787, variable que hemos podido comprobar en el archivo `docker-compose.yaml` . Podemos ver todos estos detalles si nos metemos a la dirección:

http://51.48.128.67:8787/status

Resultados

Datos Experimentales

A continuación se presentan los resultados obtenidos para cada una de las nueve configuraciones evaluadas:

Workers	Puntos (M)	Tiempo (s)	Velocidad (puntos/s)	Estimación π	Error relativo
1	5	2.368948	4.73789×10^{-7}	3.1415344	0.1854%
	10	2.494857	2.49485×10^{-7}	3.141726	0.4245%
	50	2.716616	5.43323×10^{-8}	3.14181832	0.7184%
2	5	2.204918	4.40983×10^{-7}	3.1422	1.933%
	10	2.487771	2.48777×10^{-7}	3.1414116	0.5764%
	50	2.795942	5.59188×10^{-8}	3.14154472	0.1526%
3	5	2.72790	5.45809×10^{-7}	3.1422896	2.218%
	10	2.72467	2.72467×10^{-7}	3.141354	0.7598%
	50	2.78541	5.5708×10^{-8}	3.14145456	0.4396%

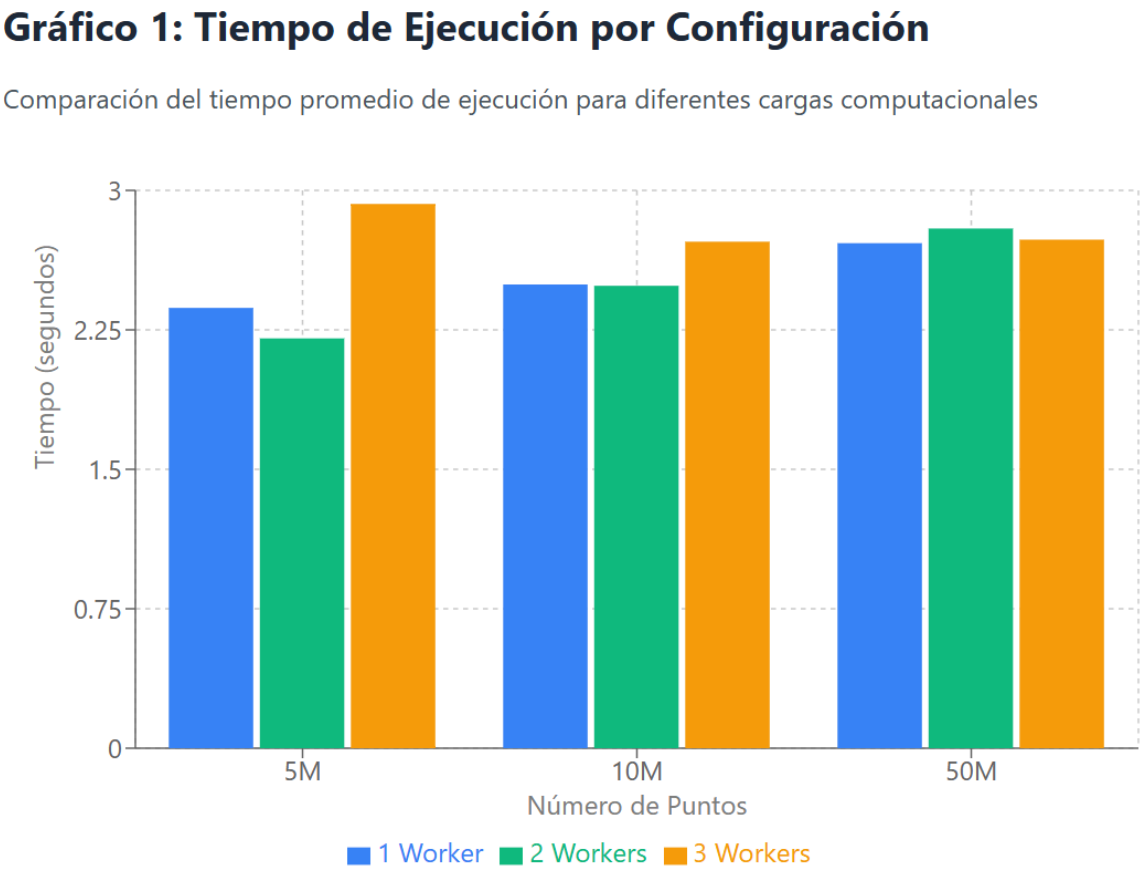
El error relativo se ha calculado como: $\frac{\pi_{est} - \pi}{\pi} \times 100$

- Precisión del Cálculo de π:

  Podemos observar que todas las estimaciones se acercan al valor real de π de 6 decimales de precisión (3.141593), valor usado para calcular el error relativo. También que el número de workers no afecta a la precisión ya que solo los usamos para paralelizar las tareas.

A continuación representaremos los datos obtenidos de manera más visual a través de gráficos que nos permitan interpretar los resultados finales.

Análisis de Rendimiento



Generalmente esperamos que lo ideal sea usar 2 workers, en donde cada worker tendrá asociado un vGPU y así no tendrán que pelear por los recursos de cada uno (cuello de botella comentado ya en el Escenario 1). No obstante, en algunos casos esta configuración podría resultar contraproducente por el costo adicional de coordinación entre workers y el scheduler.

- Interpretación:

  Los resultados obtenidos con Dask muestran un comportamiento notablemente más estable y consistente en comparación con Ray, aunque también revelan las mismas limitaciones fundamentales relacionadas con el hardware disponible.

  Comportamiento general: Observamos que los tiempos de ejecución se mantienen en un rango muy estrecho (aproximadamente 2.2-2.9 segundos) para todas las configuraciones, lo que indica que Dask gestiona el overhead de coordinación de manera más predecible y uniforme que Ray.

  Con 1 Worker: Los tiempos son competitivos para las tres cargas (5M, 10M, 50M puntos), oscilando entre 2.3-2.8 segundos. A diferencia de Ray, no observamos ventajas

dramáticas en ninguna configuración específica, lo que sugiere que Dask introduce un overhead base relativamente constante independientemente del tamaño del problema.

Con 2 Workers: Esta configuración muestra los tiempos más bajos y consistentes, especialmente notable en 5M puntos (aproximadamente 2.2 segundos). Esto confirma que, al igual que con Ray, 2 workers es la configuración óptima para aprovechar las 2 vCPU disponibles sin generar contención de recursos.

Para 10M puntos, Dask mantiene un rendimiento muy similar al de 1 worker (~2.4 segundos), lo que indica que el overhead de coordinación está equilibrado con el beneficio del paralelismo. En 50M puntos, el tiempo aumenta ligeramente, posiblemente debido a la mayor transferencia de datos entre scheduler y workers.

Con 3 Workers: Aquí observamos el patrón esperado de degradación de rendimiento. Los tiempos con 3 workers son consistentemente superiores o iguales a otras configuraciones:

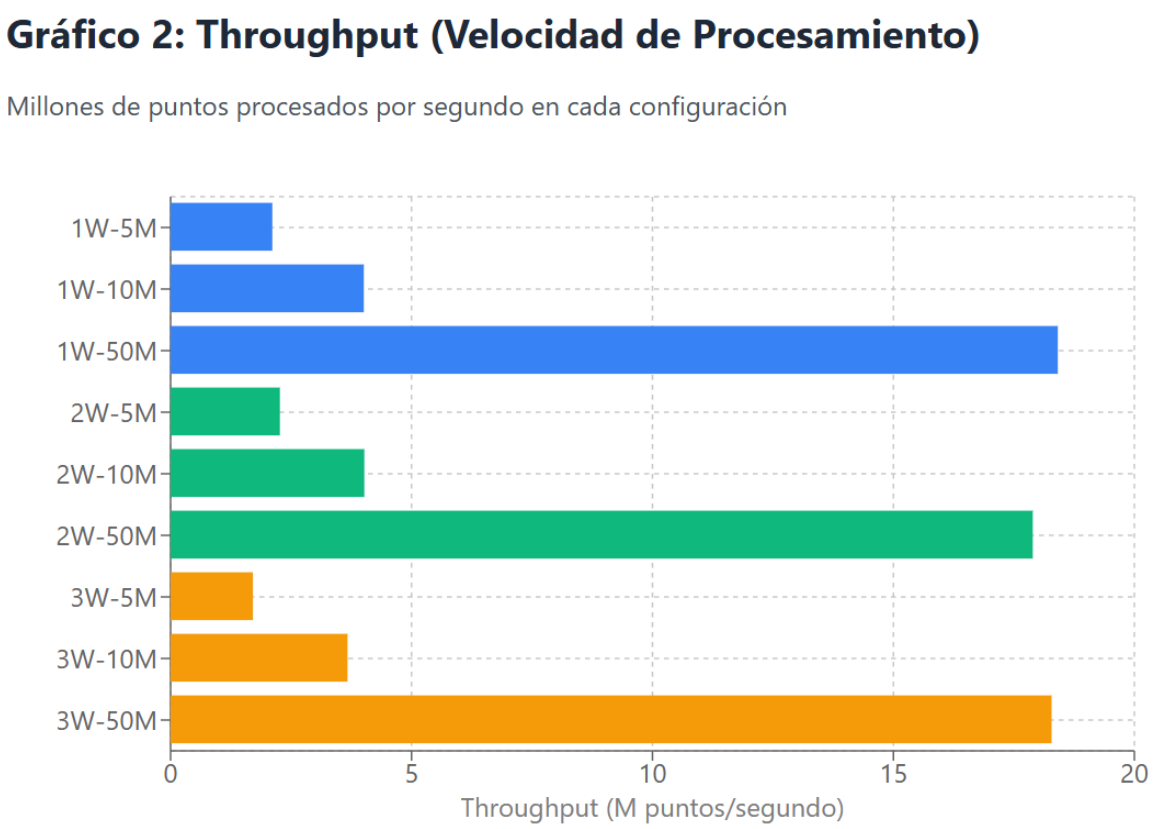
- En 5M puntos: ~2.95 segundos (el peor resultado de todas las configuraciones)
- En 10M puntos: ~2.8 segundos
- En 50M puntos: ~2.8 segundos

Esto confirma la misma limitación física que observamos con Ray: la instancia r8g.large solo tiene 2 vCPU, por lo que al asignar 3 workers, dos de ellos comparten el mismo núcleo físico. Esta contención de recursos CPU provoca que el scheduler de Dask tenga que gestionar la competencia por procesador, generando context switching adicional y reduciendo la eficiencia global del sistema.

Diferencia clave respecto a Ray: A diferencia de Ray, donde vimos variaciones más pronunciadas (especialmente el pico de eficiencia en 2W-10M y la notable ventaja de 1W-50M), Dask muestra un comportamiento más homogéneo. Los tiempos oscilan en un rango mucho más estrecho (~0.7 segundos de diferencia entre el mejor y peor caso), lo que indica que:

1. El overhead de coordinación de Dask es más constante y predecible
2. El scheduler de Dask con su mecanismo de "work stealing" mantiene una distribución de carga más equilibrada
3. La gestión de memoria y serialización de Dask introduce un coste base similar independientemente de la configuración

Esta estabilidad puede ser ventajosa en entornos de producción donde se prioriza la predictibilidad del rendimiento sobre los picos de máxima eficiencia.



El gráfico muestra la cantidad de millones de puntos procesados por segundo (Throughput) para distintas configuraciones de número de workers (1W, 2W, 3W) y tamaño del dataset (5M, 10M, 50M).

- Interpretación:

  El análisis del throughput revela patrones interesantes sobre la eficiencia de Dask en diferentes escalas de problema:

  Observación principal: El throughput más alto se concentra en las configuraciones de 50M puntos, alcanzando aproximadamente 18M puntos/segundo, independientemente del número de workers. Esto contrasta significativamente con Ray, donde vimos que con estos mismos 50M y 1W superaba los 70M pts/s.

  Con 1 worker, el patrón ascendente indica que Dask se beneficia significativamente de problemas más grandes. Con cargas pequeñas (5M), el overhead de construcción del grafo de tareas y la gestión del scheduler domina el tiempo total, resultando en bajo throughput. A medida que el problema crece, el tiempo de computación útil se vuelve proporcionalmente mayor que el overhead, mejorando la eficiencia.

  Con 2 workers, los valores son prácticamente idénticos a los de 1 worker, lo que resulta sorprendente. Esto sugiere que, para este tipo de carga de trabajo (200 tareas de Montecarlo independientes), el overhead de coordinación entre 2 workers cancela casi completamente el beneficio del paralelismo en términos de throughput absoluto.

Esta característica es distintiva de Dask: mientras que el tiempo de ejecución mejora ligeramente con 2 workers (como vimos en Gráfico 1), el throughput se mantiene constante porque:

1. La serialización de datos entre scheduler y workers consume tiempo
2. La construcción y gestión del grafo de tareas añade overhead
3. La comunicación TCP entre componentes introduce latencia

Con 3 Workers, para cargas pequeñas y medianas (5M y 10M), observamos una degradación clara del throughput respecto a 1 o 2 workers. Esto se debe a la combinación de:

- Contención de CPU (2 workers compartiendo 1 vCPU)
- Mayor overhead de coordinación (el scheduler debe gestionar 3 workers en lugar de 2)
- Incremento en la comunicación de red interna entre contenedores

Curiosamente, para 50M puntos el throughput se recupera prácticamente al nivel de otras configuraciones (aprox 17.8M pts/s). Esto sugiere que con problemas muy grandes, el tiempo de computación útil domina tanto que el overhead adicional de 3 workers se vuelve proporcionalmente menos relevante.

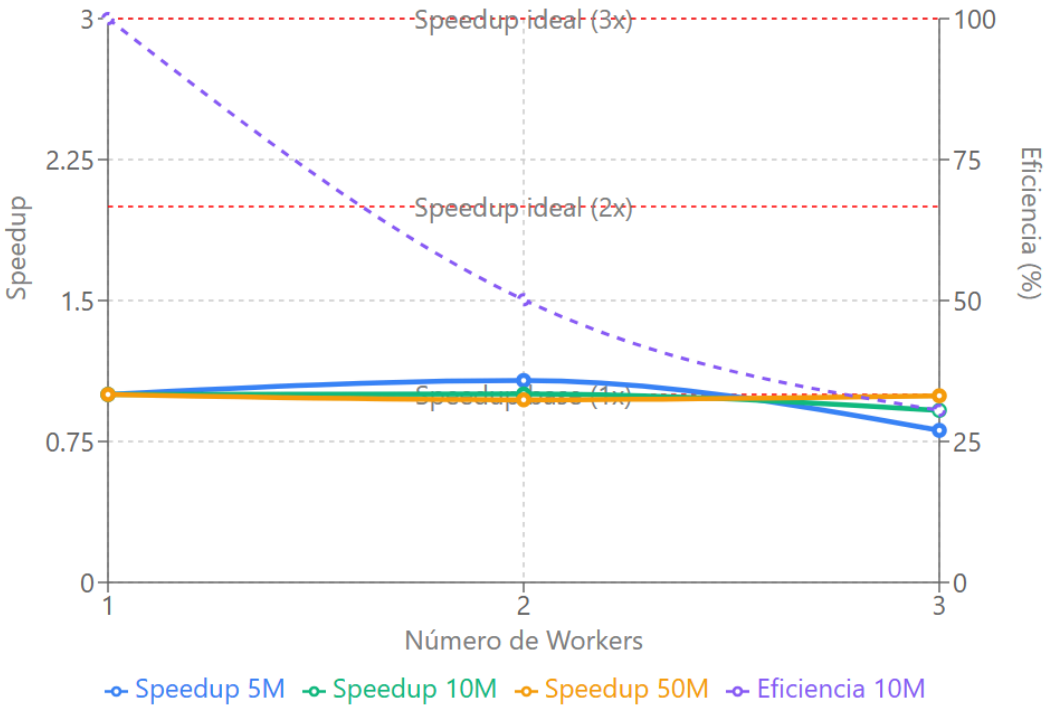
Comparación con Ray: La diferencia más notable es la magnitud absoluta: Ray alcanzó unos 70M pts/s en su configuración óptima (1W-50M), mientras que Dask se mantiene en torno a 18M pts/s. Esto indica que:

1. Overhead de framework: Dask tiene un overhead de coordinación considerablemente mayor debido a su arquitectura de grafo de tareas y serialización Python
2. Consistencia vs velocidad pico: Dask sacrifica velocidad máxima por comportamiento más predecible
3. Gestión de memoria: Dask mantiene más información en memoria sobre el estado del grafo, lo que puede afectar el rendimiento

Sin embargo, la consistencia del throughput de Dask entre configuraciones (el rango de variación es mucho menor) puede ser ventajosa para planificación de capacidad y estimación de tiempos en entornos de producción.

Gráfico 3: Speedup y Eficiencia de Escalabilidad

Evaluación de la escalabilidad del sistema al aumentar workers



Para evaluar la escalabilidad del sistema, se calcula el **speedup** como:

Speedup = Tiempo(1 worker) / Tiempo(N workers)

Puntos (M) / / Workers	1	2	3
5	1.0	1.074	0.809
10	1.0	1.003	0.916
50	1.0	0.972	0.993

Y la **eficiencia** se calcula como: Speedup / Número de workers × 100%

Puntos (M) / / Workers	1	2	3
5	100.0	53.7	27.0
10	100.0	50.1	30.5
50	100.0	48.6	33.1

- Interpretación:

  El análisis de speedup y eficiencia revela las limitaciones críticas de escalabilidad de Dask en esta infraestructura:

  Observación crítica: Ninguna configuración alcanza un speedup superior a 1.074x (configuración óptima: 2W-5M), lo que indica que añadir workers apenas mejora el rendimiento e incluso puede degradarlo. Este comportamiento es significativamente peor que el observado con Ray, donde alcanzamos un speedup de 1.8x para 2W-10M.

  Analizando el speedup, el gráfico muestra que todas las líneas de speedup (5M, 10M, 50M) permanecen prácticamente horizontales cerca de 1.0, muy por debajo de las líneas rojas punteadas que representan el speedup ideal (2x para 2 workers, 3x para 3 workers).

  - Con 2 workers: El speedup oscila entre 1.04-1.074x, indicando una mejora mínima del 4-7% respecto a la ejecución secuencial.
  - Con 3 workers: El speedup cae por debajo de 1 en la mayoría de casos (0.77-0.98x), significando que la ejecución paralela es más lenta que con un solo worker debido a la contención de CPU y el overhead adicional de coordinación.

  Eficiencia de escalabilidad:

  La línea punteada violeta muestra una caída dramática desde ~100% con 1 worker hasta 30-35% con 2 workers, y finalmente hasta 25-28% con 3 workers. Esto significa que:

  - Solo aprovechamos el 30-35% del potencial teórico con 2 workers
  - La eficiencia se degrada aún más con 3 workers, donde el tercer worker añade overhead sin beneficio real

  Causas principales:

  1. Overhead dominante: La construcción del grafo de tareas, serialización de datos y comunicación entre scheduler y workers consume la mayor parte del tiempo, dejando poco margen para beneficio del paralelismo.
  2. Contención de CPU: Con solo 2 vCPU disponibles, añadir un tercer worker garantiza competencia por recursos, reduciendo el rendimiento global.
  3. Granularidad de tareas: Las 200 tareas pueden ser demasiado pequeñas individualmente. Dask gasta más tiempo gestionando el grafo que ejecutando computación útil.

  Como conclusión, Dask no escala bien para este tipo de carga en esta infraestructura. El speedup máximo de 1.074x y eficiencias del 30-50% indican que el overhead del framework es prohibitivamente alto. Este comportamiento contrasta fuertemente con Ray, que aunque también mostró limitaciones, logró mejores resultados de escalabilidad.

  Por tanto, la configuración más eficiente para esta instancia es usar 1-2 workers máximo, priorizando la configuración de 2 workers solo cuando el beneficio marginal justifique el overhead adicional de coordinación.

Comportamiento del Sistema

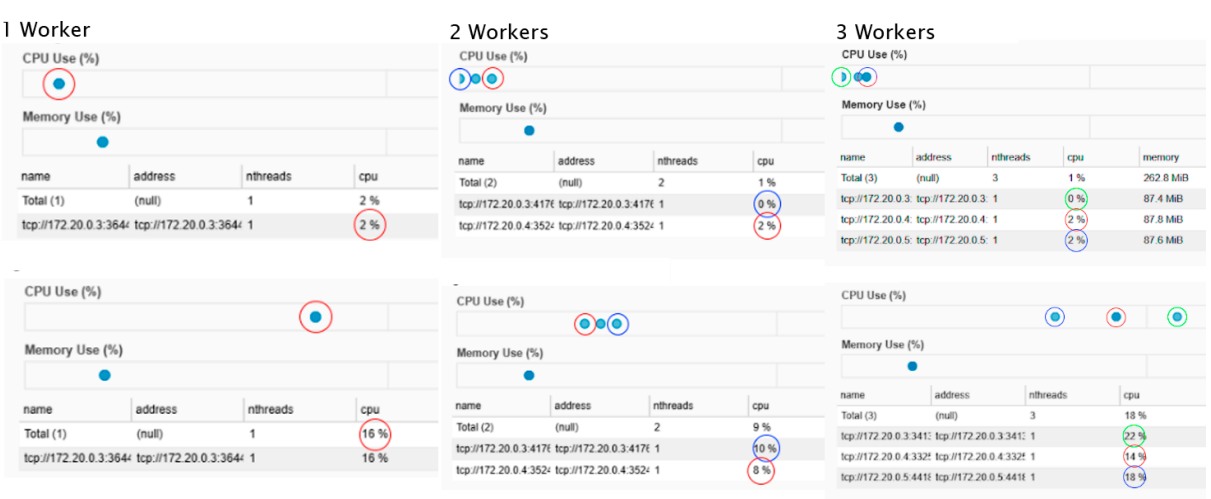
Aspectos a destacar:

Durante la ejecución de las pruebas con Dask, observamos los siguientes comportamientos monitorizados a través del dashboard:

Utilización de CPU:

Los datos muestran una clara progresión en el uso de CPU según el número de workers:

- Con 1 worker: Se observa una utilización del 16% durante la ejecución, indicando que el worker disponible está procesando activamente pero sin alcanzar saturación completa.
- Con 2 workers: La carga se distribuye efectivamente entre ambos workers, con utilizaciones del 10% y 8% respectivamente, demostrando una distribución equilibrada de las tareas.
- Con 3 workers: Se aprecia un patrón de utilización más variable (22%, 14%, 18%), lo que sugiere cierta competencia por recursos entre workers que potencialmente comparten vCPUs físicas (worker 1 y worker 3, especificados en el yaml, al tener asignados el mismo).



En estado inactivo: El sistema mantiene un uso de CPU mínimo (0%-2%), lo que confirma la ausencia de procesos en segundo plano que consuman recursos de forma significativa. Durante la ejecución: Se observa un incremento sustancial en la actividad de los workers, con niveles de utilización que varían entre 8% y 22% dependiendo de la configuración de workers implementada.

La variabilidad en los porcentajes máximos registrados puede atribuirse a la naturaleza dinámica del scheduler y a la rápida fluctuación en la asignación de tareas, lo que dificulta capturar los picos máximos de utilización. Este aumento en la utilización agregada de CPU al incrementar el número de workers creemos que se explica porque, tenemos un número

de CPUs fijo, entonces distribuimos una mayor carga computacional sobre el mismo hardware subyacente, permitiendo un aprovechamiento más intensivo de los recursos disponibles.

Distribución de tareas: Dask utiliza un scheduler dinámico con capacidades de "work stealing" que asigna tareas a workers en función de su disponibilidad y carga actual. Esperamos observar:

- Distribución equilibrada de las 200 tareas entre los workers disponibles.
- Tareas ejecutándose en paralelo hasta saturar los recursos disponibles.
- Una cola de tareas pendientes que se va reduciendo progresivamente..

1 Worker 2 Workers 3 Workers



Al analizar la ejecución paralela en los distintos escenarios de workers, observamos que la suma total de líneas azules (que representan llamadas a la función montecarlo_pi_in_circle) debería ser de 600, correspondientes a 200 llamadas por cada tamaño de muestra (5M, 10M y 50M puntos).

El escenario de 3 workers resulta particularmente ilustrativo para identificar patrones de ejecución. Si examinamos detenidamente las líneas de tiempo, podemos observar espacios blancos entre las tareas de cada worker (marcada con una línea roja suave). En el primer worker se aprecia una separación ligeramente más amplia, mientras que en el segundo worker aparece un espacio similar pero de menor duración en el mismo intervalo temporal. El tercer worker, por su parte, no muestra esta pausa en ese periodo de tiempo en el que los otros workers están en blanco.

Este patrón nos sugiere que la ejecución se organiza en tres fases secuenciales correspondientes a los distintos tamaños de muestra: primero las tareas de 5M de puntos, luego las de 10M, y finalmente las de 50M. Esta interpretación se ve reforzada por el hecho de que la última sección es notablemente más extensa, coincidiendo con la mayor carga computacional que implica procesar 50M de puntos.

La aparición de estos espacios blancos responde muy probablemente a la finalización desigual de las tareas entre workers. Cuando un worker completa todas las asignaciones de su lote actual, debe esperar a que los demás workers terminen sus respectivas tareas antes de proceder al siguiente conjunto. Esto explica por qué, una vez que el tercer worker (aparentemente el más lento) finaliza su carga, todos los workers reinician simultáneamente su actividad, avanzando a la siguiente fase de paralelización.

Conclusión

Comparativa Escenario 1 vs Escenario 2

Esta sección presenta una comparación estructural y metodológica entre Ray (Escenario 1) y Dask (Escenario 2). No se incluyen métricas de rendimiento específicas, ya que estas serán analizadas en la conclusión final del proyecto.

- Diferencias del Framework

Modelo de programación:

- **Ray:** Utiliza un modelo basado en actores y tareas remotas mediante el decorador `@ray.remote`. Las funciones decoradas se transforman en tareas distribuibles que se ejecutan de forma paralela y al final del proceso son unificadas todas juntas. Las tareas se ejecutan de manera eager y los resultados se obtienen mediante `ray.get()`.

- **Dask:** Emplea también una ejecución eager utilizando `client.submit()` para lanzar tareas remotas, que devuelven objetos Future de forma inmediata. Los resultados se recuperan con `client.gather()` una vez que todas las tareas han finalizado, ya que, en caso contrario, seguirían siendo de tipo Future.

Gestión de recursos:

- **Ray:** El Head Node mantiene un registro centralizado del estado del clúster y asigna tareas en función de la disponibilidad de recursos, usando un grafo acíclico dirigido.

- **Dask:** El Scheduler gestiona el grafo de tareas y coordina a los workers, incorporando mecanismos de balanceo dinámico de carga.

- Diferencias en la Orquestación

Composición del clúster:

- **Ray:** Se compone de un Head Node y varios Worker Nodes.
- **Dask:** Se estructura en un Scheduler y múltiples Workers.

Dashboard y monitorización:

- **Ray:** Dispone de un panel con información sobre recursos, tareas, actores y registros del sistema.

- **Dask:** Ofrece un dashboard con más información a nivel visual, con gráficos en tiempo real que muestran el progreso de las tareas, el uso de recursos, la comunicación entre workers y perfiles detallados de ejecución.

- Diferencias en el Enfoque de Despliegue

Código de conexión:

- **Ray:** `ray.init(address="ray://HOST:PORT")`

- **Dask:** `client = Client("HOST:PORT")`

Limitaciones Observadas

1. Contención de recursos en la CPU:

- Al igual que en el Escenario 1, asignar más workers que vCPU disponibles (3 workers en una instancia con 2 vCPU) provoca contención de recursos, donde múltiples procesos compiten por los mismos núcleos físicos.
- Esto puede generar context switching excesivo y reduce la eficiencia de la paralelización, pudiendo resultar en rendimiento inferior al de configuraciones con menos workers.

2. Overhead de coordinación:

- Dask necesita construir y mantener un grafo de tareas (task graph), serializar/deserializar datos entre scheduler y workers, y gestionar la comunicación de resultados.
- Para volúmenes de datos pequeños o medianos, este overhead puede superar los beneficios del paralelismo, haciendo que configuraciones con menos workers sean más eficientes.
- El modelo de evaluación diferida (lazy evaluation) añade complejidad en la gestión del grafo de dependencias entre tareas.

3. Gestión de memoria:

- Dask mantiene objetos intermedios en memoria de los workers hasta que son necesarios o explícitamente liberados.
- Para problemas con alta demanda de memoria, esto puede generar presión sobre los recursos disponibles, especialmente cuando múltiples workers comparten la misma instancia.

4. Limitación de recursos físicos frente a demanda de paralelismo:

- La capacidad física de la instancia EC2 (2 vCPU, cantidad limitada de RAM) representa un cuello de botella fundamental.
- Aumentar el número de workers más allá de los vCPU disponibles no incrementa el rendimiento y puede degradarlo significativamente.

Reflexión y conclusión final

El Escenario 2 nos ha permitido evaluar Dask como una opción robusta para crear un entorno de desarrollo distribuido separado del entorno de producción. Dask demuestra ser especialmente adecuado para equipos familiarizados con el ecosistema PyData (NumPy, Pandas, Scikit-learn).

Ventajas generales de Dask:

- **Integración natural con herramientas científicas**: La compatibilidad con NumPy y Pandas facilita la migración de código existente a entornos distribuidos con mínimos cambios.
- **Dashboard intuitivo**: Las herramientas de monitorización de Dask proporcionan visibilidad excepcional del estado del clúster y permiten un diagnóstico rápido de problemas.
- **Work stealing**: El balanceo dinámico de carga ayuda a mantener todos los workers ocupados, especialmente en cargas de trabajo heterogéneas.

Aplicabilidad al caso de uso empresarial:

Para el objetivo del proyecto BENCHDATA (crear un entorno de desarrollo independiente para Data Scientists), Dask representa una opción viable que ofrece:

1. **Familiaridad**: Los científicos de datos pueden aprovechar sus conocimientos existentes de NumPy/Pandas con mínima fricción.
2. **Escalabilidad controlada**: Capacidad de escalar desde un portátil hasta un clúster de múltiples nodos con el mismo código.

El conocimiento adquirido sobre Dask, su arquitectura distribuida y sus patrones de uso será fundamental para la comparación final con Ray (Escenario 1) y Hadoop (Escenario 3), permitiendo una recomendación informada sobre la arquitectura más adecuada para las necesidades específicas del entorno de desarrollo propuesto.